

YURI DONIZETE

Desenvolvedor Full Stack

Perdões, Minas Gerais, Brasil. Disponível para trabalho remoto.

+55 (35) 99919-6082 | yuridonizete303@gmail.com | linkedin.com/in/yuridonizete | github.com/yuri-dzt

RESUMO

Desenvolvedor full stack há 3 anos, construindo SaaS para mais de 100 clínicas odontológicas. Trabalho do banco de dados ao servidor: Node.js, TypeScript, React, PostgreSQL, IA aplicada e infraestrutura Linux. Num dos geradores com IA que fiz, o custo por geração caiu de mais de R\$ 1,00 para menos de R\$ 0,01 e o tempo de 3 minutos para 30 segundos.

EXPERIÊNCIA

Grupo Plataforma em Rede

Desenvolvedor Full Stack

Março 2023 a hoje

Perdões, MG (híbrido)

- Trabalhei em todas as camadas de uma plataforma interna com 20 aplicações, usada por mais de 100 clínicas e consultórios odontológicos. Participei desde o levantamento até a operação em produção.
- Defini a arquitetura padrão dos back-ends da plataforma, em Clean Architecture e DDD com Fastify. As camadas domain, application e infrastructure ficaram separadas, e as regras críticas passaram a ter testes automatizados.
- Decidi construir o CRM internamente depois que nenhuma opção de mercado atendeu ao processo de vendas da empresa, e ao lado dele mantenho o sistema de cadastro e gestão da base de clientes. APIs em Node.js, TypeScript e Express, dados em PostgreSQL e MySQL, interfaces em React e Next.js.
- Escolhi construir o gerador de planos de ação sobre API de LLM em vez de modelo fixo, porque o plano precisa variar por clínica. A partir de um formulário estruturado, ele produz o plano principal e um plano individual para cada colaborador.
- Desenvolvi a ZolveTV, aplicação para TVs LG de salas de espera. Os desafios foram performance em hardware limitado e execução contínua sem ninguém operar a tela.
- Criei geradores de conteúdo com IA para redes sociais e automatizei processos internos com N8N e Python, tirando da equipe rotinas manuais recorrentes.
- Assumi o deploy e a administração do servidor Linux (Ubuntu) onde as aplicações rodam: acesso por SSH, deploy containerizado com Docker Compose, Nginx como proxy reverso, certificados SSL com Certbot, rotinas agendadas no cron e gestão de DNS e CDN no Cloudflare.
- Fechei o acesso público aos serviços de dados do ambiente de produção, com mais de 30 containers em duas VMs. Os bancos passaram a ser alcançáveis apenas pelo próprio host, com senha obrigatória no Redis e acesso SSH por chave. Feito com snapshot e rollback testado, sem indisponibilidade.

PROJETOS PROFISSIONAIS

Sistema de perguntas e respostas sobre base de conhecimento (RAG) | Node.js, TypeScript, Fastify,

PostgreSQL, pgvector, React

- Construí o pipeline de ingestão: parse de PDF, chunking, geração de embeddings e gravação dos vetores em PostgreSQL com pgvector. A resposta final cita a página de origem.
- Escolhi pgvector dentro do PostgreSQL que já usávamos, em vez de um banco vetorial dedicado, para não colocar mais um serviço em operação e manter vetor e metadado na mesma consulta.
- Reduzi a taxa de fallback de 60% para 10%. A avaliação automatizada mostrou que a recuperação trazia as páginas certas, e que o gargalo estava no prompt.
- Escrevi 70 testes unitários que rodam sem banco e sem chave de API. Os provedores de embedding e de LLM podem ser trocados sem mexer nas regras de negócio.

Sistema de controle de ponto e banco de horas | Node.js, TypeScript, Fastify, Prisma, MinIO, React

- Automatizei a apuração mensal com regras de jornada, intervalos, dias irregulares e tolerância por batida. O sistema substituiu uma ferramenta terceirizada e a conferência manual da equipe, com economia de R\$ 1.400 por ano em licença.
- Implementei a gestão de afastamentos com anexos em object storage. Excluir um afastamento apaga também o arquivo, sem deixar registro órfão.
- Cobri os cenários de cálculo com 68 testes, incluindo as bordas de entrada, saída e dupla marcação.

Gerador de apresentação comercial com simulação de imagem por IA | *Node.js, TypeScript, Fastify, API de IA generativa, React*

- A partir de uma foto e dos dados do procedimento, o sistema monta um deck comercial de 10 slides para uso durante o atendimento.
- Integrei IA generativa para produzir a simulação visual do resultado esperado e os textos de cada slide.

Gerador automatizado de carrossel para redes sociais | *Python, Docker, APIs de LLM*

- Estruturei um pipeline em três camadas independentes: coleta de dados, geração de copy com estrutura AIDA e renderização visual. Uma API própria orquestra as três.
- A saída são imagens no formato nativo do Instagram (1080×1350), prontas para publicar, e tudo roda em container.
- Derrubei o custo por carrossel gerado de mais de R\$ 1,00 para menos de R\$ 0,01 e o tempo de 3 minutos para 30 segundos, com ajuste de prompt e mudanças na plataforma.

Infraestrutura compartilhada da plataforma interna | *React, Node.js, TypeScript, Fastify, Prisma, JWT, Docker*

- Criei o design system em React, com tokens semânticos e dark mode, que hoje é usado por todos os produtos da plataforma.
- Centralizei a autenticação em um serviço próprio com JWT. Antes cada produto tinha o seu login.
- Padronizei o template de projeto: a API e o front buildado sobem em um único processo, uma porta e um container. Criar e publicar uma aplicação nova ficou bem mais rápido.

PROJETOS PESSOAIS

Bloom, plataforma SaaS multi-tenant white-label | *Next.js 16, React 19, bloom-saas.vercel.app TypeScript, Prisma*

- Cada organização tem portal próprio, com identidade visual, biblioteca e membros isolados, acessível por subdomínio ou por domínio personalizado.
- Isolei os tenants em duas barreiras. O identificador da organização vem do host e nunca do cliente, e o Row Level Security do PostgreSQL funciona como segunda camada.
- Modelei módulos e planos como dado, não como código, então habilitar um recurso para uma organização não exige mexer na aplicação. As decisões de arquitetura estão registradas em ADRs.

HeyChef, cardápio digital e pedidos por QR Code | *Node.js, TypeScript, Express, Prisma, PostgreSQL, React, JWT*

- SaaS para restaurantes. O cliente abre o cardápio pelo QR Code da mesa e o pedido vai direto para a cozinha.
- Back-end em Clean Architecture com quatro camadas, validação com Zod, autenticação JWT, rate limiting e documentação da API em Swagger.

COMPETÊNCIAS TÉCNICAS

- Back-end:** Node.js, TypeScript, Fastify, Express, APIs REST, Prisma, JWT, Zod
- Front-end:** React, Next.js, Tailwind CSS, React Native
- IA aplicada:** RAG, embeddings, busca vetorial, APIs de LLM (OpenAI, Anthropic), engenharia de prompt
- Bancos de dados:** PostgreSQL, pgvector, MySQL
- Infra e DevOps:** Linux (Ubuntu), Docker, Docker Compose, Nginx, SSL/Certbot, Cloudflare, MinIO, cron, SSH
- Práticas:** Clean Architecture, DDD, testes automatizados, segurança e hardening, Git, code review, Scrum
- Linguagens e automação:** JavaScript, Python, SQL, HTML, CSS, N8N

FORMAÇÃO

- Unilavras Centro Universitário** **Janeiro 2023 a fevereiro 2025**
Tecnólogo em Análise e Desenvolvimento de Sistemas *Lavras, MG*

CERTIFICAÇÕES

- Desenvolvimento de Software Back-End**, Unilavras, 240 horas (novembro de 2024)
- Desenvolvimento Front-End**, Unilavras, 220 horas (março de 2024)
- Desenvolvimento Mobile**, Unilavras, 160 horas (janeiro de 2025)

IDIOMAS

- Português:** nativo. **Inglês:** intermediário (B1/B2), leitura de documentação e conteúdo técnico.